



Fuzzy Test

- Prerequisites:
 - Python > 3.8.2

1. Get the OpenAPI Swagger spec yaml

Generate the swagger spec yaml [openapi.yaml](#)

2. Run Fuzzy Test

2.1: Prerequisites (Validated on Ubuntu24.04)

- Install [.NET 8.0](#), for your appropriate OS:

```
# Install .NET
$ export INSTALL_DIR=$HOME/Software/dotnet
$ wget https://builds.dotnet.microsoft.com/dotnet/Sdk/8.0.414/dotnet-sdk-8.0.414-linux-x64.tar.gz
$ mkdir -p $INSTALL_DIR && tar zxf dotnet-sdk-8.0.414-linux-x64.tar.gz -C $INSTALL_DIR
$ export DOTNET_ROOT=$INSTALL_DIR
$ export PATH=$PATH:$INSTALL_DIR
$ dotnet -info
$ sudo apt-get install -y dotnet-runtime-8.0
```

- Prepare python venv

```
$ python -m venv fuzzy-venv
$ source fuzzy-venv/bin/activate
```

- Install RESTler

```
$ git clone https://github.com/microsoft/restler-fuzzer.git
$ cd restler-fuzzer
$ mkdir $HOME/Software/FuzzyTestTool/restler-bin
$ python ./build-restler.py --dest_dir $HOME/Software/FuzzyTestTool/restler-bin
```

Note: if you get nuget error NU1403 when building, a quick workaround is to clear your cache with this command: `dotnet nuget locals all --clear`

2.2: Generate RESTler Grammar

Generate RESTler Grammar:

Use RESTler to generate a grammar file from your OpenAPI spec file. Open a command prompt and run:

```
$ export PATH=$PATH:$HOME/Software/FuzzyTestTool/restler-bin/restler
$ export PYTHONPATH=$PYTHONPATH:$HOME/Software/FuzzyTestTool/restler-bin
$ mkdir outputs && cd outputs
$ Restler compile --api_spec ../../docs/user-guide/api-docs/openapi.yaml
```

This will generate a Compile directory containing the grammar files: `./Compile`

```
Compile
├─ config.json
├─ custom_value_gen_template.py
├─ defaultDict.json
├─ dependencies_debug.json
├─ dependencies.json
├─ dict.json
├─ engine_settings.json
├─ grammar.json
├─ grammar.py
├─ preprocessed
│   └─ openapi_preprocessed.json
├─ restler-20251201-015111.log
├─ StdErr.txt
├─ StdOut.txt
└─ unresolved_dependencies.json
```

2 directories, 14 files

2.3: Customizing the Dictionary

RESTler might generate random strings for your video URL, which will likely fail validation immediately. To make the fuzzing more effective:

1. Edit the `dict.json` file generated in (`./restler_output/Compile/dict.json`) to provide valid example values for specific parameters.

For examples:


```

# ...existing code...
primitives.restler_static_string("""
    "video":"""),
primitives.restler_custom_payload("video", quoted=True),
primitives.restler_static_string("""
    "prompt":"""),
primitives.restler_custom_payload("prompt", quoted=True),
primitives.restler_static_string("""
    "method":"""),
primitives.restler_fuzzable_group("method", ['SIMPLE', 'USE_VLM_T-1', 'USE_LLM_T-1', 'USE_...
primitives.restler_static_string("""
    "processor_kwargs":
        {
            "process_fps":"""),
primitives.restler_custom_payload("process_fps"),
primitives.restler_static_string("""
            "levels":"""),
primitives.restler_custom_payload("levels"),
primitives.restler_static_string("""
            "level_sizes":"""),
primitives.restler_custom_payload("level_sizes"),
primitives.restler_static_string("""
            "chunking_method":"""),
primitives.restler_fuzzable_group("chunking_method", ['pelt', 'uniform'] , default_enum
# ...existing code...

```

2.4: Run Fuzzy Tests

More details can be found at [Restler-fuzzer Tutorial](#).

Fuzz test

Start with a lightweight test:

```
$ Restler test --grammar_file Compile/grammar.py --dictionary_file Compile/dict.json --set
```

Fuzz lean

Here is an example:

```
$ Restler fuzz-lean --grammar_file Compile/grammar.py --dictionary_file Compile/dict.json
```

Analyze Results

RESTler creates a Test or FuzzLean directory inside your current working directory (e.g., `RestlerResults/experiment...`).

- `logs/main.txt` : The main log file.
- `bug_buckets/` : Contains specific request/response sequences that caused 500 errors or other failures.

Post-analysis with custom rules

If you found any bugs in reports, and want to verify it after fixing, can use replay to re-test using the following command:

```
# Analyze existing results and apply custom rules
Restler replay --replay_log FuzzLean/RestlerResults/experiment30025/bug_buckets/InvalidVal

# --replay_log <path to the RESTler bug bucket repro file or trace database>.
#     The replay log file extension may be either '.replay.txt' or '.ndjson'.
```

Note: after cleaning all bugs, you should re-generate the fuzzy test using:

```
Restler fuzz-lean ...
```